

CUDA Jacobi 集项目验收讲解稿

本文档用于现场验收时给老师讲解项目。整体顺序与最终报告保持一致，但表述改为现场讲解口吻，并在需要现场演示的位置给出测试命令、预期输出和解说词。

0. 现场准备

讲解目标

验收时重点说明三件事：

- (1) 本项目完成了课程要求中的论文理解、串行代码对照分析、CUDA 并行实现和结果验证。
- (2) 并行化不是简单把代码改成 .cu 文件，而是把串行算法中“逐边 Jacobi 判定”这个核心独立循环提取出来，映射为 GPU 的一线程一边。
- (3) 程序不仅能跑，而且通过了预处理、GPU 计算、退化边 SoS 回退、串行并行结果比对、性能测试和可视化验证。

现场目录

验收时先进入并行代码目录：

```
cd "/mnt/d/金介然/大三下/gpu/大作业/CODE/mycode"
```

我的解说词：

老师，我现场演示主要在 CODE/mycode 目录下进行。这里是我自己开发的 CUDA 并行版本，串行参考工程放在 CODE/JacobiSetComputation-master，报告和分析文档放在 docs/analysis。

1. 先说明课程要求和项目完成情况

讲解内容

课程要求主要有四项：

- (1) 翻译并整理 Jacobi Sets of Multiple Morse Functions 论文。

- (2) 对照论文思路和串行项目，开发对应 CUDA 并行代码。
- (3) 详细分析代码实现思路，结合论文算法理解说明实现细节。
- (4) 把论文翻译、CUDA 核心代码、代码实现思路分析整理到一个文档中。

我的解说词：

本项目围绕论文中的 Jacobi 集算法展开。串行项目已经实现了二维三角网格上两个 Morse 函数的 Jacobi 集计算。我做的工作是先理解串行实现，再把其中最适合并行化的逐边判定部分改写为 CUDA kernel，同时保留 CPU 端的拓扑预处理和 SoS 精确回退，最后用测试脚本验证结果和串行版本一致。

最终报告分成三个部分：第一部分是论文翻译，第二部分列出 CUDA 并行核心代码，第三部分详细分析串行逻辑、论文算法映射、CUDA 实现、测试结果和性能表现。

2. 串行算法核心思路讲解

2.1 串行工程结构

讲解时可以打开或指出以下文件：

```
CODE/JacobiSetComputation-master/src/main.cpp
CODE/JacobiSetComputation-master/include/TriMeshJ.h
CODE/JacobiSetComputation-master/src/TriMeshJ.cpp
CODE/JacobiSetComputation-master/include/JacobiSet.h
CODE/JacobiSetComputation-master/src/JacobiSet.cpp
```

我的解说词：

串行工程主要分成两层。第一层是网格拓扑层，由 `TriMeshJ` 负责读取三角网格、生成边集合、建立顶点邻接，并为每条边找到 link 中的两个对顶点。第二层是 Jacobi 判定层，由 `JacobiSet` 负责实现 `POS1`、`POS2`、`is_lowerLink`、`alignment` 和 `compute()`。

对二维三角网格而言，一条内部边通常被两个三角形共享。边的两个端点记为 `e1`, `e2`，两个相邻三角形中不在边上的顶点就是 `link1`, `link2`。Jacobi 判定只需要当前边的两个端点、两个 link 顶点，以及这四个顶点上的两个函数值。

2.2 串行核心循环

讲解重点：

```

for(edge in mesh->edges) {
    get_e_link(...);
    is_lowerLink(...);
    if (f_crit) {
        push_back Jacobi edge;
    }
}

```

我的解说词：

串行代码的核心就是遍历所有边。每条边先通过 `get_e_link()` 找到两侧的 link 顶点，然后分别判断这两个 link 顶点是否属于 lower link。若两个 link 顶点的分类满足临界关系，就把这条边加入 Jacobi 集。

这里最重要的观察是：每条边的判定只依赖它自己的局部信息，不依赖其他边的计算结果。所以这些边可以独立计算，这就是我后面做 CUDA 并行化的依据。

3. 并行化设计讲解

3.1 为什么选择“边”为并行粒度

我的解说词：

这个算法最适合并行化的部分不是文件读取，也不是网格邻接表构建，而是 `JacobiSet::compute()` 中的逐边判定循环。因为边数随着网格规模增大而线性增长，每条边的计算流程相同，而且边与边之间没有数据依赖。

所以我采用的 CUDA 映射是：内部边数组中的第 `i` 条边，对应 CUDA 中第 `i` 个线程。每个线程读取一条 `EdgeRecord`，执行与串行版本相同的 lower-link 判定，最后把结果写入 `results[i]`。

3.2 CPU-GPU 混合方案

讲解结构：

CPU: mesh 读取 + 拓扑预处理 + SoS 回退 + 输出

GPU: 内部边并行 Jacobi 判定

我的解说词：

本项目没有把所有逻辑都搬到 GPU。原因是 `trimesh2` 的网格结构和 SoS 符号计算库都依赖复杂的 CPU 数据结构，不适合直接放到 device 端。

因此我采用 CPU-GPU 混合方案。CPU 负责读取 mesh、建立边表、处理边界边、生成 GPU 可直接访问的线性数组；GPU 负责大规模规则的逐边判定；GPU 标记出的退化边再回到 CPU，用串行工程原有的 SoS 逻辑重新计算，保证最终结果和串行 SoS 版本一致。

4. 并行代码结构讲解

4.1 并行工程文件

讲解文件：

```
CODE/mycode/main_cuda.cpp
CODE/mycode/jacobi_cuda.cu
CODE/mycode/jacobi_gpu.h
CODE/mycode/CMakeLists.txt
CODE/mycode/build_cuda_wsl.sh
CODE/mycode/compare_jacobi.py
CODE/mycode/benchmark_jacobi.py
CODE/mycode/test_*.sh
```

我的解说词：

`main_cuda.cpp` 是 CPU 端主程序，负责读取网格、生成边表、调用 GPU、执行 SoS 回退和写出结果。

`jacobi_gpu.h` 定义 CPU 和 GPU 之间共享的数据结构，包括 `EdgeRecord`、`JacobiGpuResult` 和 `GpuTiming`。

`jacobi_cuda.cu` 是 CUDA 核心实现，里面包含 device 版的 `POS1`、`POS2`、`is_lower_link`，以及真正执行并行判定的 `compute_jacobi_kernel()`。

4.2 GPU 输入结构

核心结构：

```
struct EdgeRecord {
    int e1;
    int e2;
    int link1;
    int link2;
};
```

我的解说词：

EdgeRecord 是 GPU 计算的输入单位。它把一条内部边判定所需的局部拓扑信息压缩成四个整数：边的两个端点 `e1`, `e2`，以及 link 中的两个对顶点 `link1`, `link2`。

这样 GPU kernel 就不需要访问 `TriMesh`、邻接表或者 `std::set`，只需要按下标读取连续数组中的 `EdgeRecord`。

4.3 GPU 输出结构

核心结构：

```
struct JacobiGpuResult {
    int fv1;
    int fv2;
    int min_f;
    int min_g;
    int pos_align;
    int is_jacobi;
    int degenerate;
};
```

我的解说词：

串行程序只保存已经判定为 Jacobi 集的边，但 GPU 版本为每条内部边都保存一个结果。这样做是为了避免多个线程同时 `push_back` 造成写冲突。

每个线程只写自己的 `results[idx]`，不需要原子操作，也不需要线程间同步。`degenerate` 字段用于标记这条边是否触发近退化判断，后面 CPU SoS 回退只处理这些边。

4.4 CUDA kernel

核心逻辑：

```
const int idx = blockIdx.x * blockDim.x + threadIdx.x;
if (idx >= edge_count) {
    return;
}

const EdgeRecord edge = edges[idx];
```

我的解说词：

kernel 中的线程编号 `idx` 对应边数组中的下标。每个线程取一条边，然后执行四次 lower-link 判定：分别判断两个 link 顶点相对于两个函数的分类。

最后线程把结果写回 `results[idx]`。整个过程没有线程间依赖，这就是它适合 CUDA 并行的原因。

4.5 SoS 回退

我的解说词：

Jacobi 集计算中存在浮点退化问题，例如函数值相等、三点在 (f, g) 平面中接近共线等。串行版本用 SoS 保证符号判定稳定。

GPU 端没有完整重写 SoS，而是检测近退化情况并设置 `degenerate = 1`。回到 CPU 后，程序只对这些退化边调用串行 `JacobiSet::is_lowerLink()` 和 `alignment()` 重新计算，用 SoS 结果覆盖 GPU 临时结果。

这样普通边走 GPU 快路径，退化边走 CPU 精确路径，最终既保留并行速度，又保证结果和串行 SoS 版本一致。

5. 现场测试流程

本节是验收现场真正运行的部分。建议按顺序执行，时间不够时至少执行构建、预处理、GPU 基础计算、SoS 回退、串行并行比对。

5.1 进入项目目录

命令：

```
cd "/mnt/d/金介然/大三下/gpu/大作业/CODE/mycode"
```

我的解说词：

下面我进入 CUDA 并行版本目录，所有测试脚本都在这个目录下执行。

5.2 检查 CUDA 编译器

命令：

```
nvcc --version
```

预期结果包含：

```
Cuda compilation tools, release 12.0, V12.0.140
```

我的解说词：

这一步用于确认 WSL 中 CUDA Toolkit 可用。这里显示 `nvcc` 版本，说明 CUDA 编译器可以正常使用。

5.3 检查 GPU 运行环境

命令：

```
nvidia-smi
```

预期结果包含：

```
GPU Name: NVIDIA GeForce RTX 4060  
CUDA Version: 13.0
```

我的解说词：

这一步确认 WSL 能识别到 NVIDIA GPU。`nvidia-smi` 中的 `CUDA Version` 是驱动支持的最高运行时版本，`nvcc --version` 是实际安装的 Toolkit 版本，驱动版本高于 Toolkit 版本是正常的。

5.4 构建 CUDA 程序

命令：

```
bash build_cuda_wsl.sh
```

预期结果包含：

```
-- Configuring done
-- Generating done
[100%] Built target jacobi_cuda
Built: /mnt/d/金介然/大三下/gpu/大作业/CODE/mycode/build/jacobi_cuda
```

我的解说词：

这一步通过 CMake 构建 CUDA 程序。构建目标叫 `jacobi_cuda`。这里同时编译了我的 CUDA 代码，也链接了串行工程中的 `TriMeshJ.cpp`、`JacobiSet.cpp`、`trimesh2` 和 `SoS` 库。

构建成功说明 C++ 编译器、CUDA 编译器、静态库和 CMake 配置都能正常协同工作。

6. 功能正确性测试

6.1 CPU 预处理测试

命令：

```
bash test_preprocess.sh
```

预期结果包含：

```
vertices: 512
edges: 1536
faces: 1024
interior_edges: 1536
```

我的解说词：

这个测试只验证 CPU 端预处理，不运行 GPU kernel。程序读取 `torus` 测试网格，调用 `need_edges()` 和 `need_neighbors()`，然后对每条边调用 `get_e_link()`，生成内部边表。

输出中顶点数是 512，边数是 1536，面数是 1024，内部边数也是 1536。对这个 `torus` 网格来说，所有边都是内部边，所以 `interior_edges` 和 `edges` 一致。这说明 GPU 输入边表是正确的。

6.2 GPU 基础计算测试

命令：

```
bash test_gpu_basic.sh
```

预期结果包含：

```
jacobi_edges: 64
degenerate_edges: 16
sos_fallback_edges: 16
gpu_kernel_ms: ...
gpu_total_ms: ...
output: /tmp/test_torus_gpu_jacobi.txt
```

我的解说词：

这个测试会真正调用 CUDA 程序，GPU 对所有内部边执行并行 Jacobi 判定。

jacobi_edges: 64 表示最终得到 64 条 Jacobi 边。degenerate_edges: 16 表示 GPU 判定过程中发现 16 条近退化边。sos_fallback_edges: 16 表示这 16 条边都交给 CPU SoS 逻辑重新计算并覆盖结果。

gpu_kernel_ms 是纯 kernel 时间，gpu_total_ms 是包含显存分配、数据传输、kernel 执行和结果拷回的 GPU 调用总时间。

6.3 退化边导出测试

命令：

```
bash test_degenerate_dump.sh
```

预期结果包含：

```
degenerate_edges: 16
sos_fallback_edges: 16
degenerate_dump: /tmp/test_torus_degenerate_edges.txt
```

我的解说词：

这个测试验证 `--dump-degenerate` 功能。除了统计退化边数量，程序还会把退化边列表写到 `/tmp/test_torus_degenerate_edges.txt`。

这个文件主要用于调试，可以看到哪些边触发了近退化判断，也能说明 SoS 回退不是随便做的，而是只针对 GPU 标记出的退化边。

6.4 SoS 回退测试

命令：

```
bash test_sos_fallback.sh
```

预期结果包含：

```
jacobi_edges: 64
degenerate_edges: 16
sos_fallback_edges: 16
output: /tmp/test_torus_gpu_sos_jacobi.txt
```

我的解说词：

这个测试重点验证 CPU SoS 回退是否正常。GPU 先快速判定所有边，标记 16 条退化边，然后 CPU 用串行 SoS 逻辑重新计算这些边。

最终输出 `/tmp/test_torus_gpu_sos_jacobi.txt`，这个文件后面会和串行参考结果做比对。

7. 串行与并行结果比对

7.1 边集比对

命令：

```
bash test_compare_jacobi.sh
```

预期结果包含：

```
reference_edges: 64
candidate_edges: 64
match: yes
```

我的解说词：

这一步是正确性验证的关键。 `reference_edges` 是串行 SoS 版本输出的 Jacobi 边数量， `candidate_edges` 是 CUDA hybrid 版本输出的 Jacobi 边数量。

两者都是 64，并且 `match: yes`，说明并行版本得到的 Jacobi 边集合与串行参考完全一致。

这里使用的是边集比较，不依赖输出顺序。即使后续 GPU 端采用压缩输出导致顺序变化，只要边集合一致，测试仍然能判断正确。

7.2 文本文件直接比较

命令：

```
diff -u "../JacobiSetComputation-master/test_torus_jacobi.txt" /tmp/test_torus_gpu_sos_jacobi.t
```

预期结果：

无输出

我的解说词：

`diff` 没有输出表示两个文件内容完全一致。也就是说，在当前实现中，CUDA hybrid 版本不仅边集合和串行版本一致，连文本输出顺序也一致。

8. 性能测试

8.1 小规模冒烟测试

命令：

```
bash test_benchmark_smoke.sh
```

预期结果包含：

```
32x16: cpu_wall=... ms, gpu_wall=... ms, speedup=...x, match=yes  
wrote: /tmp/jacobi_benchmark_smoke.csv
```

我的解说词：

这个测试用于确认 benchmark 脚本本身能正常运行。它会生成一个小规模 torus 网格，分别运行串行版本和 CUDA hybrid 版本，并确认 `match=yes`。

这里小规模下 GPU 端到端可能比 CPU 慢，这是正常的，因为端到端时间包含进程启动、文件 I/O、CPU 预处理、CUDA 内存分配、数据传输和 SoS 初始化等固定开销。

8.2 多规模性能测试

命令：

```
python3 benchmark_jacobi.py --sizes 32x16,64x32,128x64,256x128,512x256 --output /tmp/jacobi_bench  
cat /tmp/jacobi_benchmark.csv
```

报告中的参考结果：

```
32x16: cpu_wall=45.496 ms, gpu_wall=273.897 ms, speedup=0.166x, match=yes  
64x32: cpu_wall=93.851 ms, gpu_wall=247.151 ms, speedup=0.380x, match=yes  
128x64: cpu_wall=190.336 ms, gpu_wall=287.831 ms, speedup=0.661x, match=yes  
256x128: cpu_wall=519.644 ms, gpu_wall=464.860 ms, speedup=1.118x, match=yes  
512x256: cpu_wall=1802.050 ms, gpu_wall=1115.213 ms, speedup=1.616x, match=yes
```

我的解说词：

多规模测试可以看到两个结论。第一，所有规模都是 `match=yes`，说明不同规模下 CUDA hybrid 输出都和串行结果一致。

第二，小规模时 GPU 端到端不占优势，因为固定开销占主导；随着网格规模增大，逐边判定数量增加，GPU 并行优势开始体现。

在报告中的测试结果里，256x128 网格开始超过串行版本，加速比约为 1.118x；到 512x256 网格时，端到端加速比达到 1.616x。

另外，GPU kernel 时间始终约 1 ms 左右，说明真正并行化的核心逐边判定在 GPU 上执行效率很高。端到端时间主要受 CPU 预处理、I/O、数据传输和 SoS 回退影响。

9. 可视化验证

9.1 生成 VTK 文件

命令：

```
python3 jacobi_to_vtk.py /tmp/test_torus_gpu_sos_jacobi.txt  
cp /tmp/test_torus_gpu_sos_jacobi.vtk "/mnt/d/金介然/大三下/gpu/大作业/CODE/mycode/test_torus_gpu_sos_jacobi.vtk"
```

预期结果：

Wrote /tmp/test_torus_gpu_sos_jacobi.vtk

我的解说词：

程序输出的 Jacobi 集是文本格式，为了在 ParaView 里查看，需要先转换成 VTK 文件。这里生成的是 test_torus_gpu_sos_jacobi.vtk。

9.2 ParaView 观察结果

讲解要点：

- (1) 单独打开 Jacobi 集 VTK 文件，可以看到两条闭合曲线。
- (2) 和 test_torus.obj 一起加载后，曲线贴在 torus 表面。
- (3) 从正面看是两条同心圆，从侧面放大看分别位于圆环内外两侧。

我的解说词：

当前测试函数是 $f=y$, $g=x$ 。在对称 torus 上，理论上 Jacobi 集会落在 $z \approx 0$ 的截面附近，所以单独看 Jacobi 集会表现为内外两条闭合圆曲线。

把 Jacobi 集和 torus 曲面一起加载后，可以看到红色曲线贴在网格表面，没有明显偏移或断裂。这说明并程序输出的结果不仅文本上和串行一致，几何位置上也符合预期。

10. 代码实现总结

10.1 本项目完成的核心工作

我的解说词：

总结一下，本项目完成了四件核心工作。

第一，理解论文和串行代码，把 Jacobi 集判定落实到二维三角网格中的逐边 lower-link 判定。

第二，提取出适合 GPU 并行的核心循环，也就是对所有内部边进行独立判定，并设计 EdgeRecord 作为 GPU 输入结构。

第三，在 CUDA 中实现 device 版的 POS1、POS2 和 is_lower_link，用一个线程处理一条边，避免线程间写冲突。

第四，针对浮点退化问题保留 CPU SoS 回退，使最终结果能够与串行 SoS 版本完全一致。

10.2 项目取舍

我的解说词：

本项目没有追求把所有逻辑都放到 GPU，而是做了异构划分。网格读取、邻接结构、SoS 符号计算这些复杂逻辑保留在 CPU；规则、重复、无依赖的大规模边判定放到 GPU。

这个取舍的好处是工程实现更稳定，也更容易和串行参考对齐。测试结果表明，在结果正确的前提下，大规模网格上已经能体现 GPU 并行优势。

11. 现场验收推荐顺序

如果现场时间充足，按以下顺序完整演示：

```
cd "/mnt/d/金介然/大三下/gpu/大作业/CODE/mycode"
nvcc --version
nvidia-smi
bash build_cuda_ws1.sh
bash test_preprocess.sh
bash test_gpu_basic.sh
bash test_degenerate_dump.sh
bash test_sos_fallback.sh
bash test_compare_jacobi.sh
diff -u "../JacobiSetComputation-master/test_torus_jacobi.txt" /tmp/test_torus_gpu_sos_jacobi.txt
bash test_benchmark_smoke.sh
python3 benchmark_jacobi.py --sizes 32x16,64x32,128x64,256x128,512x256 --output /tmp/jacobi_benchmark.csv
cat /tmp/jacobi_benchmark.csv
python3 jacobi_to_vtk.py /tmp/test_torus_gpu_sos_jacobi.txt
```

如果现场时间有限，至少演示以下命令：

```
cd "/mnt/d/金介然/大三下/gpu/大作业/CODE/mycode"
bash build_cuda_wsl.sh
bash test_preprocess.sh
bash test_gpu_basic.sh
bash test_sos_fallback.sh
bash test_compare_jacobi.sh
```

我的解说词：

如果时间有限，我重点展示构建、预处理、GPU 计算、SoS 回退和串行并行结果比对。这几步已经能证明程序可以正常构建、GPU 并行逻辑可以运行、退化边有精确回退，并且最终结果与串行参考一致。

12. 老师可能追问的问题

问题 1：为什么不把整个程序都放到 GPU？

回答：

因为整个程序里并不是所有部分都适合 GPU。网格读取是 I/O，trimesh2 使用复杂 C++ 容器，SoS 是 CPU 符号计算库，这些放到 GPU 会增加大量工程复杂度。真正适合 GPU 的是逐边 Jacobi 判定，因为它规则、数量大、无数据依赖。所以我采用 CPU-GPU 混合方案。

问题 2：GPU 版本如何保证和串行版本一致？

回答：

普通边在 GPU 上复刻串行非 SoS 的 lower-link 判定逻辑。遇到近退化情况时，GPU 不直接信任浮点结果，而是标记 degenerate，然后 CPU 调用串行工程原有的 SoS 逻辑重新计算并覆盖结果。最后通过 compare_jacobi.py 和 diff 验证输出与串行 SoS 版本一致。

问题 3：为什么 GPU 小规模时反而慢？

回答：

报告里的 gpu_wall_ms 是端到端时间，包含文件读取、CPU 预处理、CUDA 内存分配、数据传输、SoS 初始化和文件输出。小规模时真正需要并行计算的边数不多，固定开销占主导，所以端到端不加速。随着网格规模增大，逐边判定数量增多，GPU 并行优势才开始体现。

问题 4：代码中最核心的并行点是哪一段？

回答：

最核心的是 `jacobi_cuda.cu` 里的 `compute_jacobi_kernel()`。它用 `idx = blockIdx.x * blockDim.x + threadIdx.x` 得到线程对应的边下标，每个线程读取一条 `EdgeRecord`，执行四次 lower-link 判定，然后把结果写入 `results[idx]`。

问题 5：SoS 回退会不会抵消 GPU 加速？

回答：

一般不会，因为 SoS 回退只处理 GPU 标记出的退化边，而不是重新计算所有边。在 `torus` 测试中，1536 条内部边中只有 16 条退化边。大多数普通边仍然由 GPU 并行完成，SoS 回退主要用于保证正确性。

13. 结束语

我的解说词：

综上，本项目完成了从论文算法理解、串行代码分析到 CUDA 并行实现的完整流程。并行化的核心是把每条内部边的 Jacobi 判定映射到一个 GPU 线程，同时用 CPU 端 SoS 回退保证退化情况下的正确性。

从测试结果看，CUDA hybrid 版本在标准 `torus` 示例和多规模 benchmark 中都与串行版本保持一致；在较大网格上，端到端运行时间也体现出 GPU 并行优势。因此，本项目满足课程中“对照论文思路及串行代码，开发对应并行 CUDA 代码，并详细分析实现思路”的要求。